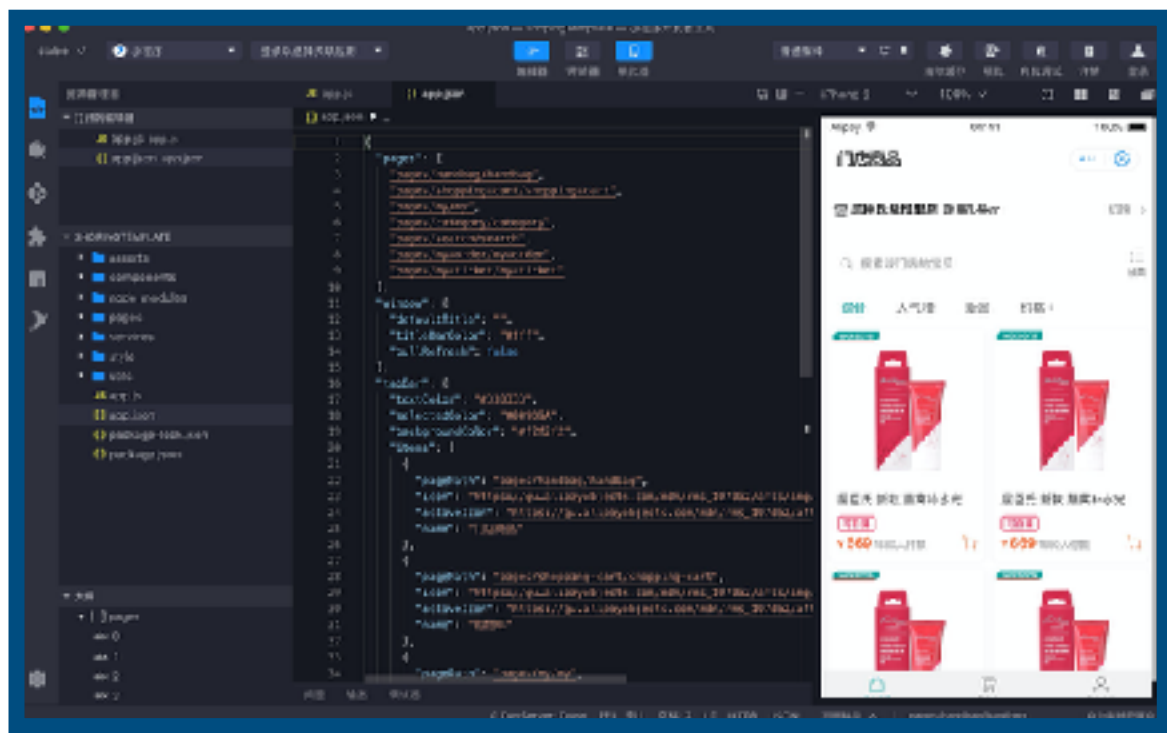


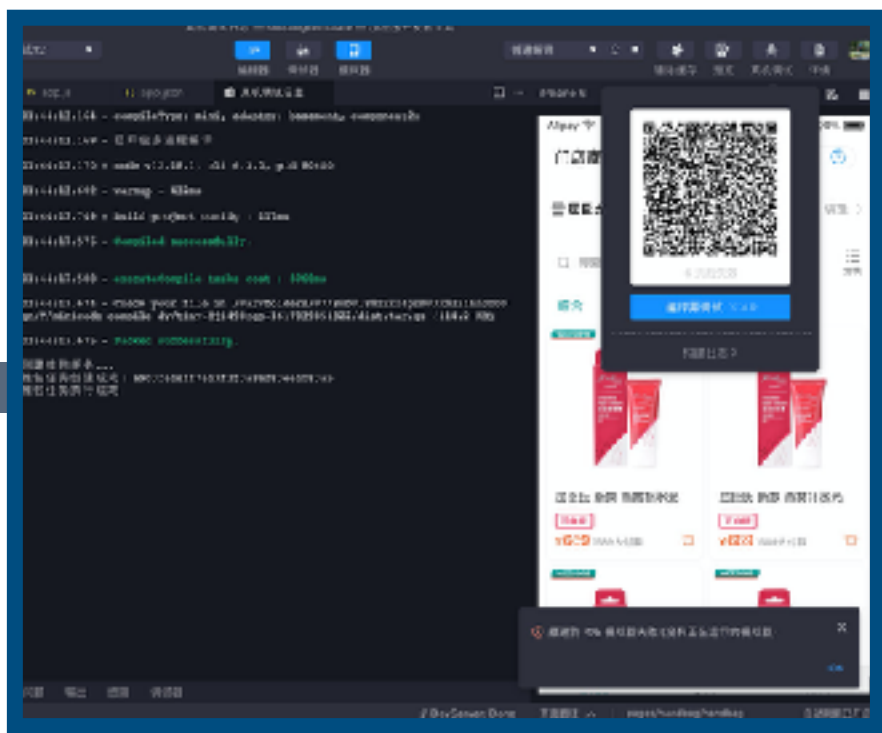
如何建立前端标准化研发流程

以蚂蚁内部小程序研发为例

你是如何研发小程序的？



编写代码



本地调试



上传版本



正式发布



上架审核



体验版测试与验证

以上的研发流程足够了么？

当前研发流程所面临的问题

从蚂蚁内部研发规范与要求看

- **能力问题**

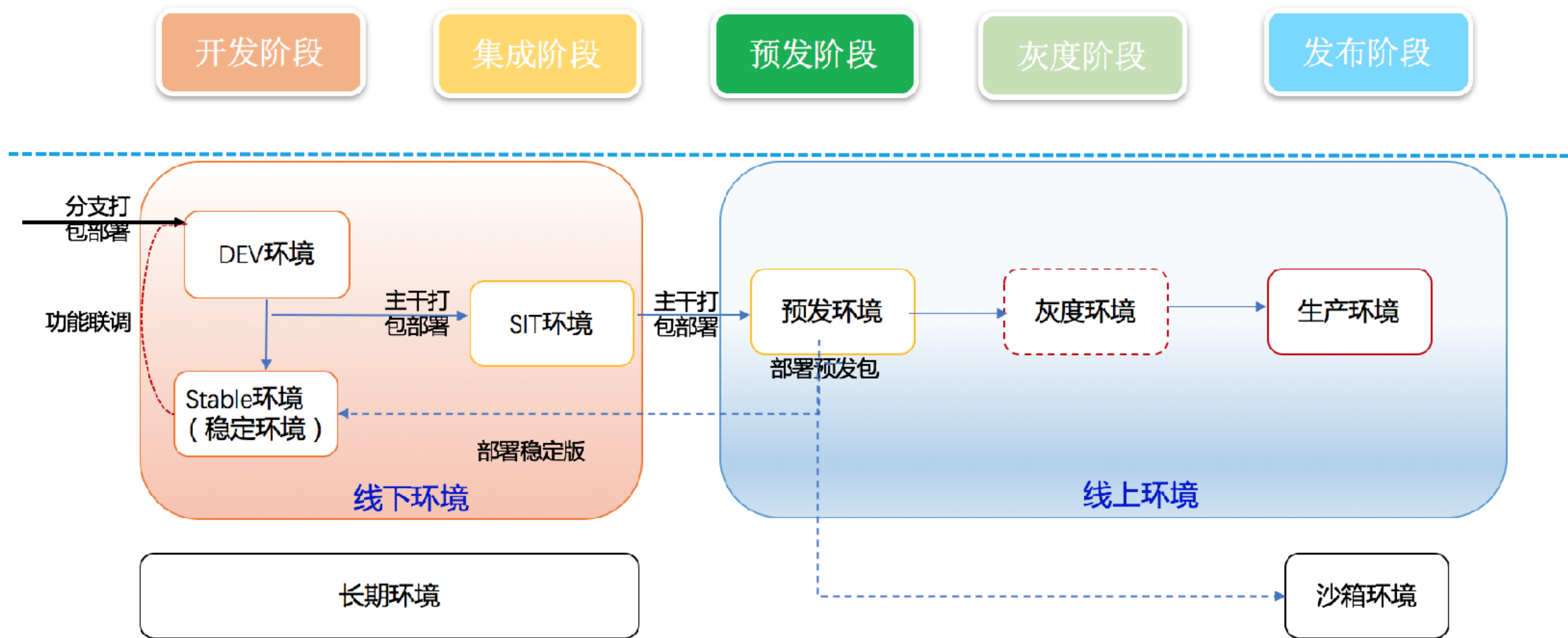
- 没有开发/测试/预发多环境区分，无法与后端进行 rpc 联调
- 在本地 IDE 中无法进行多人协同开发小程序
- 多个迭代无法同时在一个环境下部署，进行测试验证
- 没有线上监控报警能力，没有线上变更防御能力

- **体验问题**

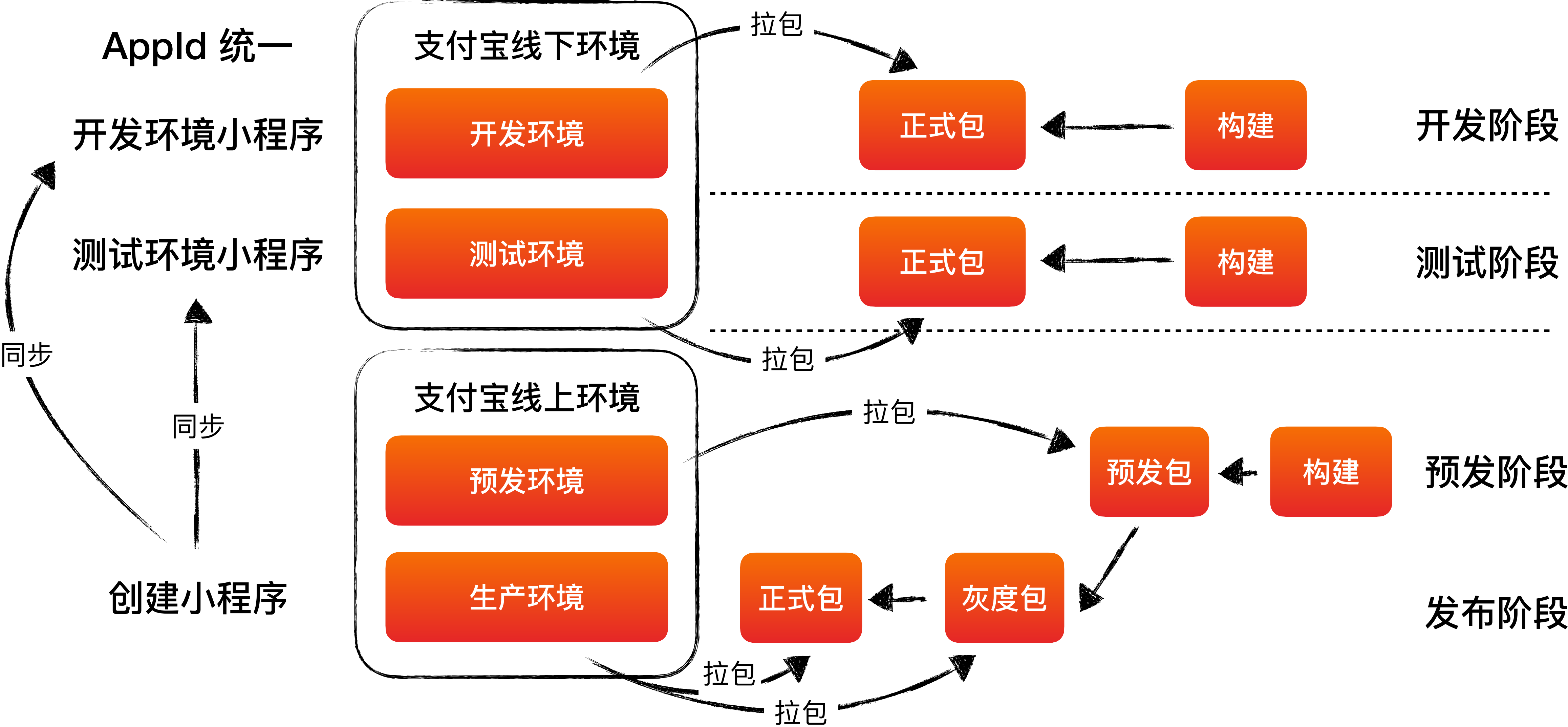
- 人工上架审核漫长，尤其是半夜的发布，找人困难
- 构建速度急需提升，CI 需要加速，达到快速部署验证的目的

研发多环境

蚂蚁研发环境区分

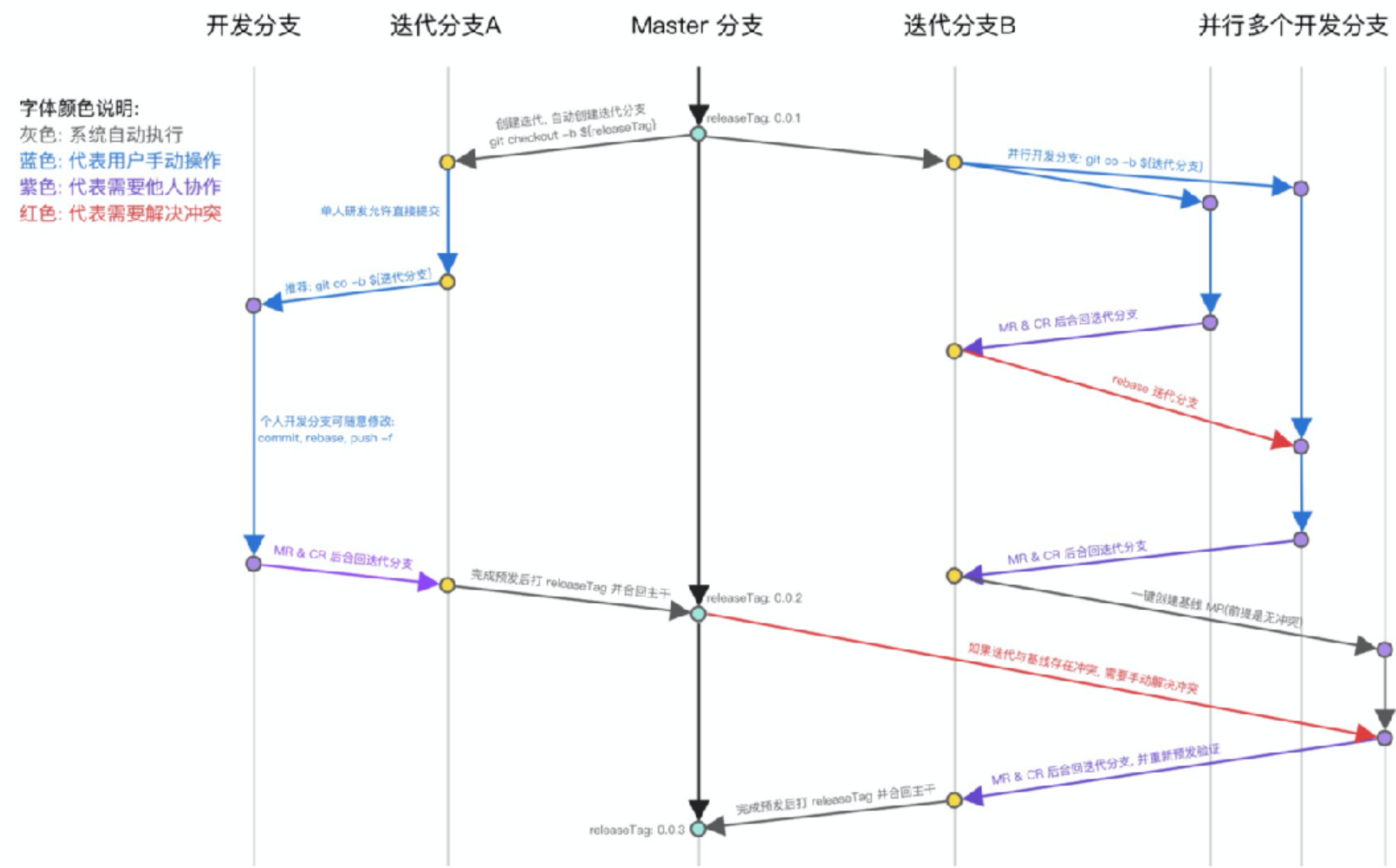


小程序研发多环境



如何进行多人研发协同?

- master 为主干分支，不允许直接提交代码
- 新的迭代分支从 master 最新 release tag checkout
- 迭代分支不允许直接提交代码，提交代码以 MR 形式合入迭代分支
- 从开发阶段进入测试阶段之后，迭代必须包含最新发布 release tag



一定要 MR 么？

- **严格模式：**

无法直接向迭代分支提交代码，必须 MR 经 code review 后执行迭代分支的代码变更

- **普通模式：**

允许开发者直接向迭代分支提交代码，但在测试阶段进入预发阶段必须要进行评审，进入预发后相当于严格模式

- **宽松模式：**

允许开发者直接向迭代分支提交代码，仅在迭代发布前会对所有变更做一次性评审

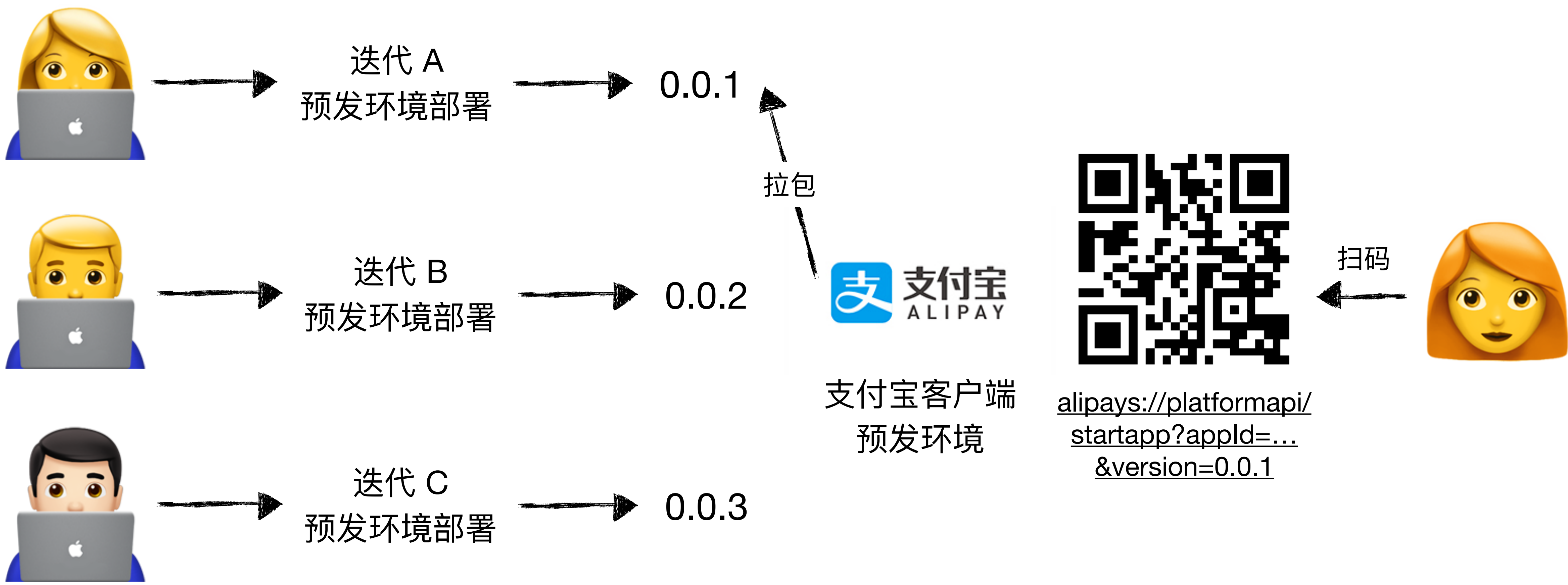
代码评审设置

代码评审模式 ?

- ☐ 严格模式：每次变更都需评审 推荐
- ☐ 普通模式：进入预发阶段前变更一次性评审，进入预发阶段后每次变更都需评审
- ☒ 宽松模式：迭代发布前所有变更一次性评审

更新

同一环境多分支部署



发布过程中的变更防御

- 变更：任何与线上环境相关的操作，包括代码变更、配置变更、数据变更等等
- 变更三板斧：可灰度、可监控、可应急
- 变更防御是什么？在变更过程中，实时监控发布情况，防范故障发生
- 为什么要做变更防御？90%以上的线上故障都是由于线上变更引起的，做好变更防御，能有效防范线上故障的发生
- 怎么做小程序变更防御？在灰度过程中，发布版本流量/异常/白屏/闪退/错误率未达标，发布熔断，不允许继续发布



变更防御

=

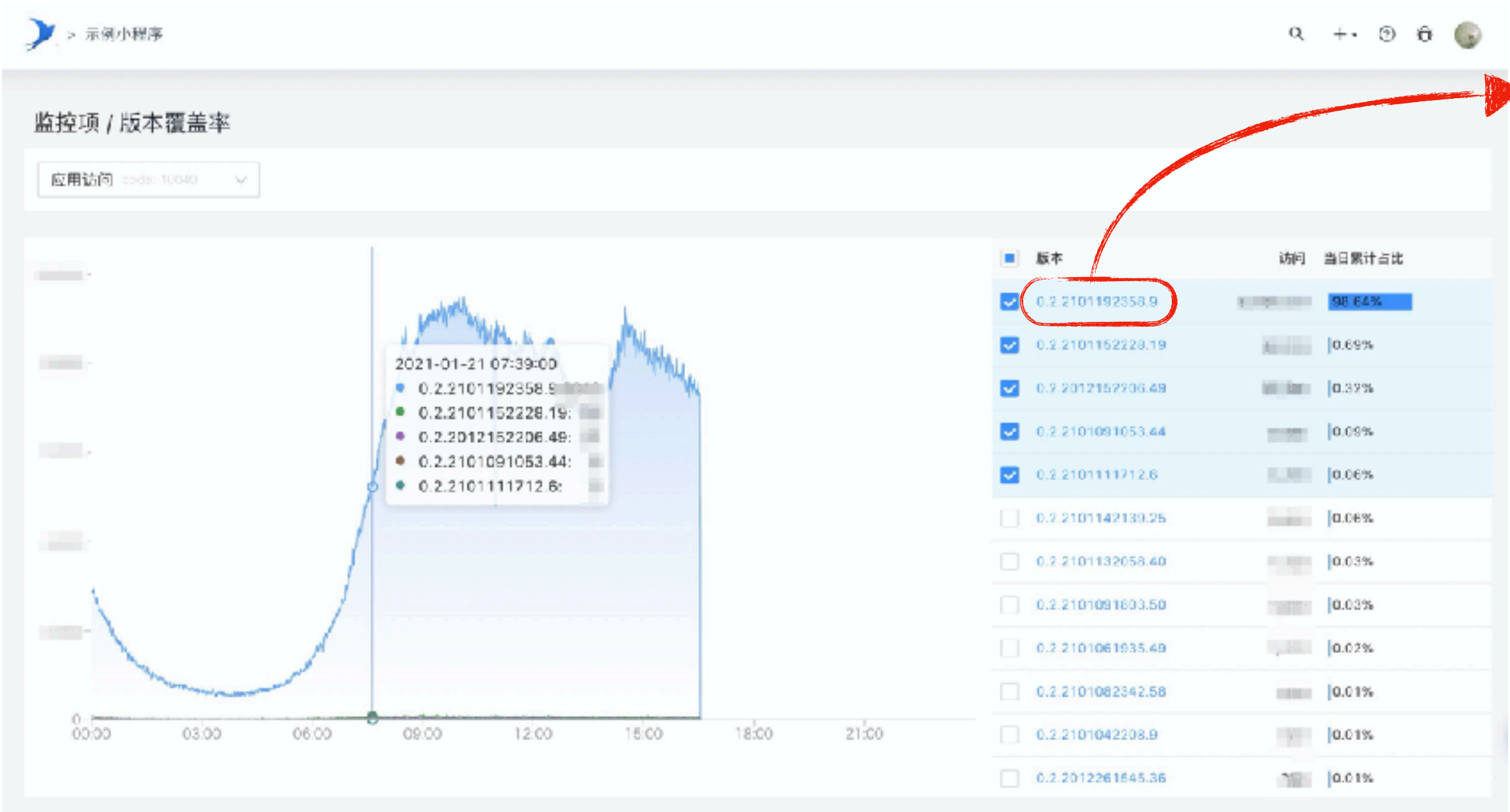
数据

×

监控

×

门禁

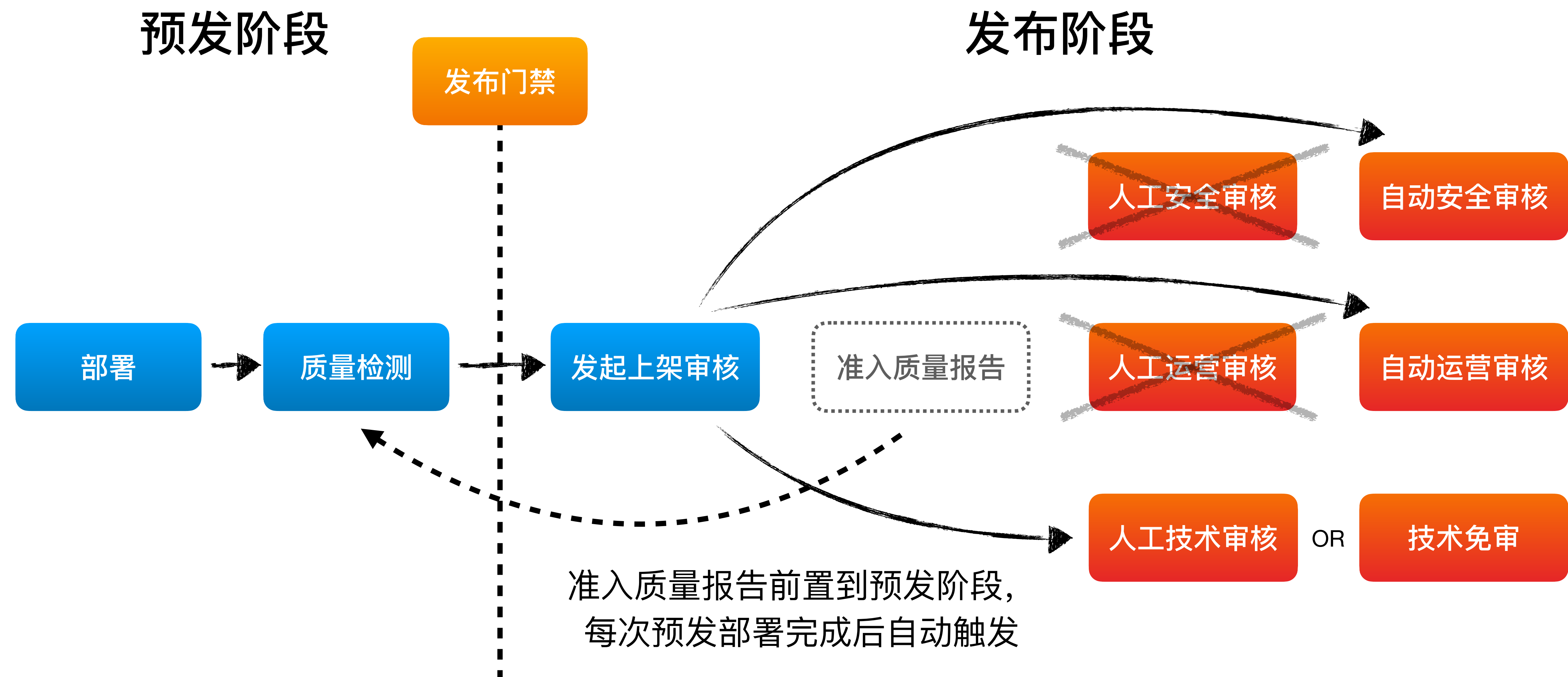


版本覆盖率



版本容器数据

小程序上架审核加速



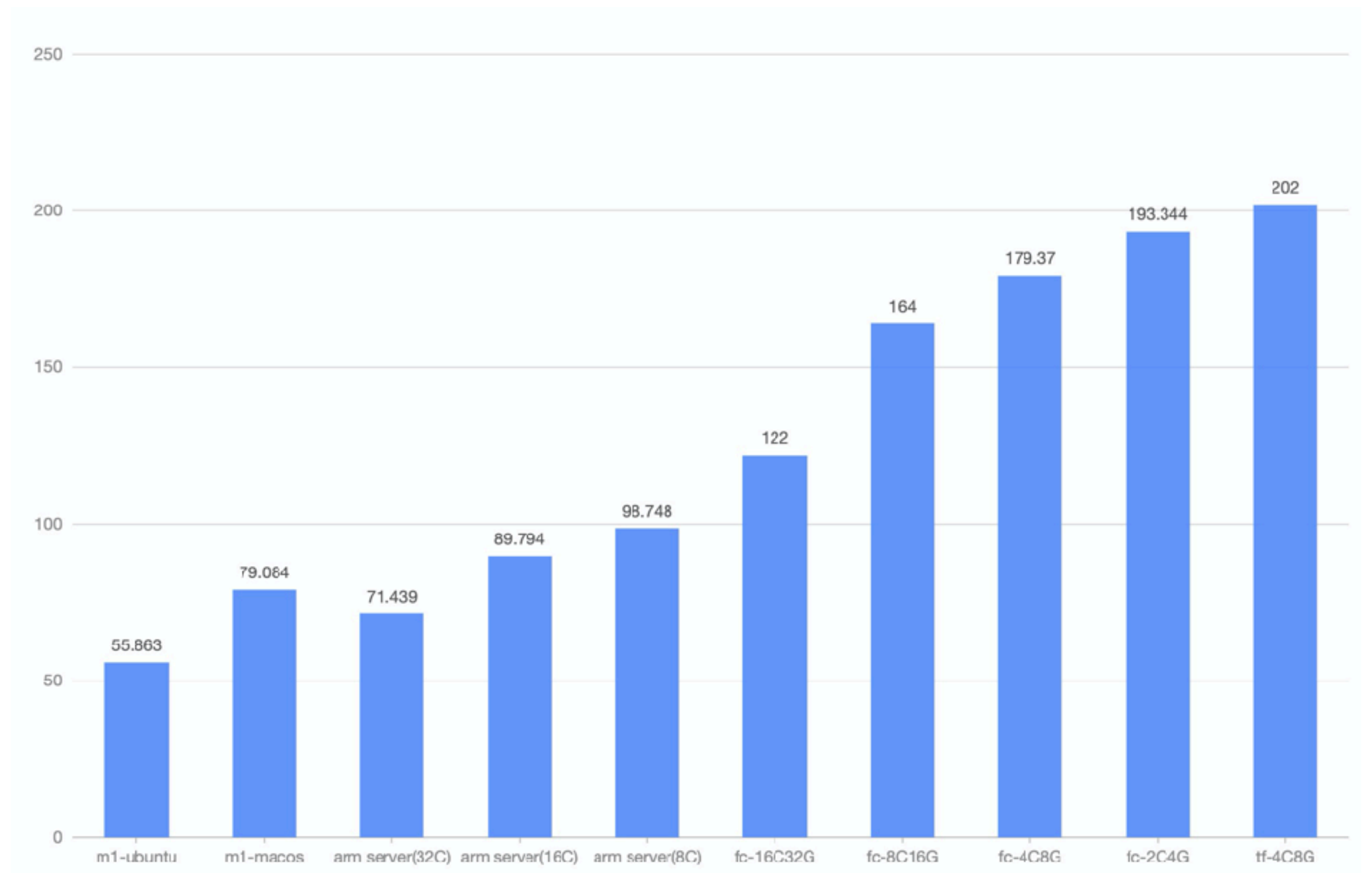
小程序构建加速

- 资源投入

- 使用阿里云 FC（函数计算），按需调整性能配置
- 探索 M1 芯片的 mac mini 搭建构建集群

- 策略优化

- 增加构建 Cache
- 多进程/并行化构建
- 长耗时任务异步化（sourcemap 构建）
- 代码变更后自动运行构建



同一小程序不同环境资源下构建 benchmark

CI/CD 相关要点回顾

- **持续集成 (CI)**

- 良好的代码管理与版本控制能力，满足多人协同
- 迭代分支代码要包含发布代码基线，防止大幅偏离主干
- CI 全程自动化，运行速度要尽可能得快，重点在构建环节进行优化

- **持续交付 (CD)**

- 多环境部署/相同环境多版本部署
- 代码单元测试/自动化集成测试/人工覆盖率测试结合

- **持续部署 (CD)**

- 发布过程中的自动化灰度放量与变更防御
- 要做到真正的自动化持续部署很难

我能去实现这些优化么？

代码版本控制



Github



Gitlab

审核加速



IDE 预审核

监控报警



开放平台云监控



第三方监控平台

自建 CI/CD



支付宝沙箱环境



MiniU

其他类型应用该怎么做呢？

蚂蚁大前端所涉及的应用类型

前端应用

Assets

混合研发

H5

静态站点

小程序

服务端应用

Chair Application

Chair Function

Chair Task

资产应用

npm 组件

闪蝶组件

小程序插件

其他

一体化应用

区块

为何要建立标准化前端研发流程？

体验统一

- 存在多种类型应用，需要标准流程来统一心智
- 开发者在不同应用类型下面对的是相对一致的研发流程

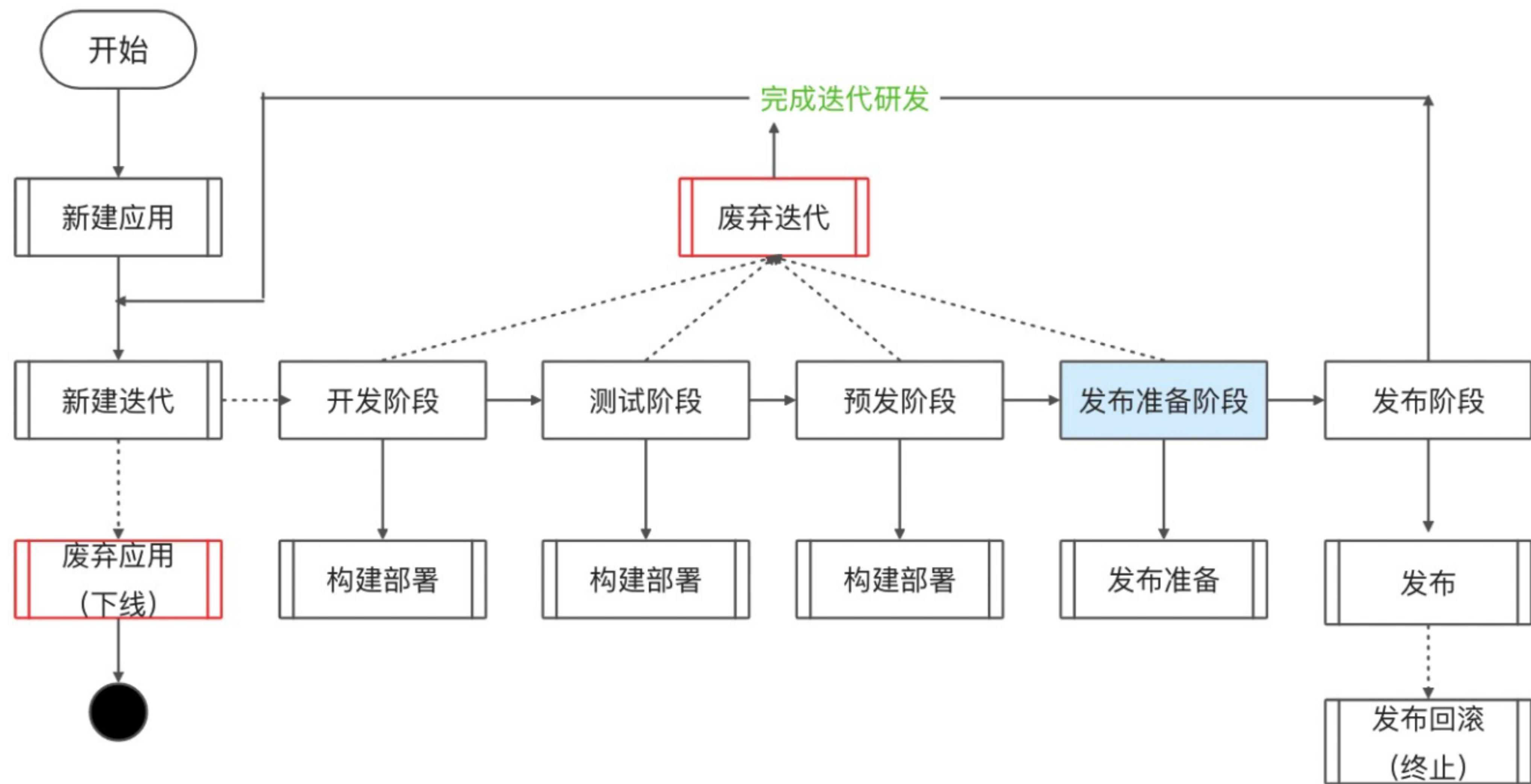
满足管控

- 研发流程须符合变更的基本要求与管控原则
- 将必要流程内置到标准流程当中，潜移默化研发者，降低学习成本

流程开放

- 有效帮助业务平台扩展出结合自身业务特点的研发流程，无需进行重复建设
- 解放研发平台生产能力，专注于能力建设与流程管控

标准化前端研发流程描述方式



定义阶段

YAML 流程编排

- 应用新建流程
- 迭代流转流程
- 构建部署流程
- 发布流程
- ...

```
environments:
  YUYAN_CHANGECORE_CONTEXT_yuyanId: "{{ yuyanId }}"
  YUYAN_CHANGECORE_CONTEXT_operatorWorkId: "{{ operatorWorkId }}"
  YUYAN_CHANGECORE_CONTEXT_env: "{{ env }}"
  YUYAN_CHANGECORE_CONTEXT_sprintId: "{{ sprintId }}"
  YUYAN_CHANGECORE_CONTEXT_commitId: "{{ commitId }}"
  YUYAN_CHANGECORE_CONTEXT_miniAppId: "{{ miniAppId }}"
  YUYAN_CHANGECORE_CONTEXT_buildVersion: "{{ buildVersion }}"

stages:
  - 构建
  - 部署

构建:
  stage: 构建
  pluginConfig:
    serviceConfig:
      result:
        name: buildArtifacts
      common:
        timeout: 3600
      action:
        retry:
          enable: false
  params:
    type: MINI_APP_BUILD

部署:
  stage: 部署
  pluginConfig:
    serviceConfig:
      result:
        name: miniAppDeploy
      common:
        timeout: 3600
      action:
        retry:
  params:
    type: CREATE_DEPLOYMENT
    templateCustomParams: "{{ templateCustomParams | dump | safe }}"
```

- 流程入参
- 流程阶段定义
- 流程任务定义
- 流程任务入参
- 流程任务超时时间

定义行为

应用元数据驱动

基础元信息

yuyanid scene

租户 repo

label 成员角色

应用生命周期

行为定义

构建模型

部署模型

发布/回滚模型

label

平台关联

构建平台

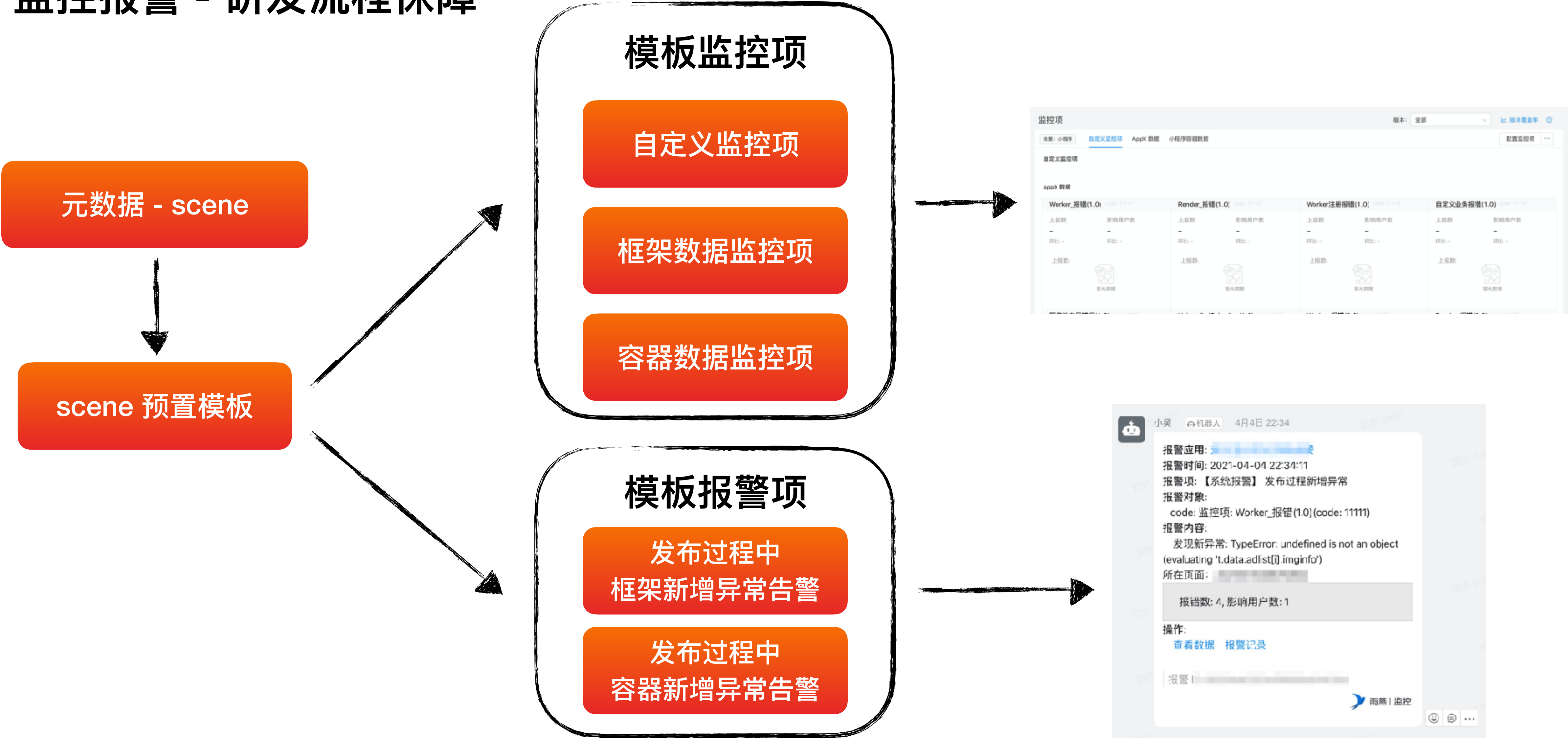
部署平台

架构工作台

高可用治理平台

标准化前端研发流程配套设施

监控报警 - 研发流程保障



标准化前端研发流程配套设施

数据大盘 - 反哺研发流程

应用维度

- 分布：类型/组织/研发方式
- 状态：是否新增/是否活跃

迭代维度

- 时长：交付周期
- 状态：是否发布

构建部署维度

- 构建/部署成功率
- 构建/部署耗时

发布维度

- 发布审核/发布灰度时长
- 灰度/发布/回滚成功率